



DIKTISAINTEK
BERDAMPAK



UNIVERSITAS
BRAWIJAYA

brone
BRAWIJAYA ONLINE LEARNING



MODUL PRAKTIKUM

CRUD dan Validasi Data pada Laravel

Pengembangan Aplikasi Web Lanjut

Pertemuan 4 | Workbook Tutorial Mandiri

Disusun oleh: Dosen Pengampu

Versi Dokumen: 1.0

Tanggal: September 2026



filkom.ub.ac.id

Cara Menggunakan Modul

Modul ini dirancang untuk dikerjakan secara mandiri. Ikuti urutan tahap dari atas ke bawah dan jangan melompat ke tahap berikutnya sebelum checkpoint pada tahap aktif berhasil.

Target akhir

Katalog Buku v4 memiliki operasi CRUD lengkap, validasi server-side, proteksi CSRF, method spoofing untuk PUT/DELETE, error feedback, old input, dan flash message.

Studi kasus	Katalog Buku dari Materi 3
Titik awal	Project Laravel sudah terhubung ke database dan dapat menampilkan data books
Mode kerja	Praktikum mandiri, langkah demi langkah
Fokus	CRUD, validation, CSRF, mass assignment, Form Request
Output	Aplikasi Katalog Buku v4 + challenge ISBN

Capaian Pembelajaran Praktikum

1. Memetakan operasi CRUD ke HTTP method, route, dan action pada resource controller.
2. Membangun create, read, update, dan delete dengan Eloquent.
3. Menerapkan CSRF protection dan method spoofing pada form Blade.
4. Menerapkan validasi input menggunakan Request validation dan Form Request.
5. Menampilkan error, old input, dan flash message sebagai feedback kepada pengguna.

Prasyarat dari Materi 3

Sebelum memulai, pastikan kondisi project Anda memenuhi checklist berikut. Jika salah satu belum terpenuhi, perbaiki terlebih dahulu sebelum masuk ke CRUD.

- [] Project Laravel dapat dijalankan dengan php artisan serve.
- [] Konfigurasi database pada .env sudah benar.
- [] Tabel books sudah tersedia dan migration sudah dijalankan.
- [] Model App\Models\Book sudah tersedia.
- [] Halaman GET /books dapat menampilkan data dari database.
- [] Kolom minimum pada books: title, author, year, created_at, updated_at.

Perintah verifikasi awal

```
php artisan migrate:status
php artisan route:list --path=books
php artisan serve
```

Peta Alur CRUD



Benang merah praktikum: Route -> Controller -> Validation -> Model -> Database -> Response

Setiap perubahan data harus melewati kontrak yang jelas: HTTP method dan route menentukan intent, controller mengorkestrasi request, validation menyaring input, Eloquent berinteraksi dengan database, lalu aplikasi mengembalikan view atau redirect.

Tahap 1 - Menyiapkan Resource Controller dan Resource Route

Laravel menyediakan resource controller untuk pola CRUD yang konsisten. Pada tahap ini, kita menyiapkan tujuh action standar dan satu resource route.

1. Buat resource controller

```
php artisan make:controller BookController --model=Book --resource
```

2. routes/web.php

```
<?php

use App\Http\Controllers\BookController;
use Illuminate\Support\Facades\Route;

Route::resource('books', BookController::class);
```

3. Verifikasi route

```
php artisan route:list --path=books
```

HTTP Method	URI	Action	Route Name
GET	/books	index	books.index
GET	/books/create	create	books.create
POST	/books	store	books.store
GET	/books/{book}	show	books.show
GET	/books/{book}/edit	edit	books.edit
PUT/PATCH	/books/{book}	update	books.update
DELETE	/books/{book}	destroy	books.destroy

Checkpoint 1

Perintah `route:list` harus menampilkan tujuh route books dengan HTTP method, URI, action, dan nama route yang sesuai.

Tahap 2 - Menyiapkan Model untuk Mass Assignment

Method `create()` dan `update()` dapat menerima array data sekaligus. Karena itu, field yang boleh diisi secara massal perlu dibatasi pada model.

`app/Models/Book.php`

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    protected $fillable = [
        'title',
        'author',
        'year',
    ];
}
```

Penting

Gunakan data hasil validasi dan whitelist field pada model. Hindari `Book::create($request->all())` untuk input yang berasal dari pengguna.

Tahap 3 - READ: Daftar dan Detail Buku

Operasi Read menggunakan dua action: `index()` untuk collection dan `show()` untuk satu model. Route model binding akan mengisi parameter Book `$book` berdasarkan `{book}` pada route.

`app/Http/Controllers/BookController.php`

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use Illuminate\Http\Request;

class BookController extends Controller
{
    public function index()
    {
        $books = Book::latest()->get();

        return view('books.index', compact('books'));
    }

    public function create()
```

```

{
    return view('books.create');
}

public function store(Request $request)
{
    // Diisi pada tahap berikutnya.
}

public function show(Book $book)
{
    return view('books.show', compact('book'));
}

public function edit(Book $book)
{
    // Diisi pada tahap update.
}

public function update(Request $request, Book $book)
{
    // Diisi pada tahap update.
}

public function destroy(Book $book)
{
    // Diisi pada tahap delete.
}
}

```

resources/views/books/index.blade.php

```

@extends('layouts.app')

@section('content')
    <h1>Daftar Buku</h1>

    <a href="{{ route('books.create') }}">Tambah Buku</a>

    @if (session('success'))
        <p>{{ session('success') }}</p>
    @endif

    <table border="1" cellpadding="8" cellspacing="0">
        <thead>
            <tr>
                <th>Judul</th>
                <th>Penulis</th>
                <th>Tahun</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody>
            @forelse ($books as $book)
                <tr>
                    <td>{{ $book->title }}</td>
                    <td>{{ $book->author }}</td>
                    <td>{{ $book->year }}</td>
                    <td>
                        <a href="{{ route('books.show', $book) }}">Detail</a>
                    </td>
                </tr>
            @empty
                <tr>
                    <td colspan="4">Belum ada data buku.</td>
                </tr>
            @endforelse
        </tbody>
    </table>

```

```

        </tr>
    @endforelse
</tbody>
</table>
@endsection

```

resources/views/books/show.blade.php

```

@extends('layouts.app')

@section('content')
    <h1>Detail Buku</h1>

    <p><strong>Judul:</strong> {{ $book->title }}</p>
    <p><strong>Penulis:</strong> {{ $book->author }}</p>
    <p><strong>Tahun:</strong> {{ $book->year }}</p>

    <a href="{{ route('books.index') }}">Kembali</a>
@endsection

```

Checkpoint 2

GET /books menampilkan daftar buku. Tautan Detail membuka GET /books/{book} dan menampilkan satu record yang benar.

Layout Blade Minimum

Jika project dari pertemuan sebelumnya sudah mempunyai layout, Anda boleh mempertahankannya. Jika belum, gunakan layout minimum berikut agar semua view pada modul dapat dijalankan.

resources/views/layouts/app.blade.php

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'Katalog Buku')</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 900px; margin: 32px auto; padding: 0 16px; }
        nav { margin-bottom: 24px; }
        a { margin-right: 8px; }
        input { display: block; width: 100%; max-width: 480px; padding: 8px; margin: 4px 0 10px; }
        .error { color: #b91c1c; }
        .success { color: #166534; }
        table { width: 100%; border-collapse: collapse; }
        th, td { text-align: left; padding: 8px; border: 1px solid #d1d5db; }
        form.inline { display: inline; }
    </style>
</head>
<body>
    <nav>
        <a href="{{ route('books.index') }}">Daftar Buku</a>
        <a href="{{ route('books.create') }}">Tambah Buku</a>
    </nav>

    @yield('content')
</body>
</html>

```

Tahap 4 - CREATE: Form Tambah dan Store

Create terdiri dari dua request yang berbeda: GET /books/create menampilkan form kosong, lalu POST /books mengirim data ke store().

Method create() dan store() pada BookController

```
public function create()
{
    return view('books.create');
}

public function store(Request $request)
{
    $validated = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'author' => ['required', 'string', 'max:150'],
        'year' => ['required', 'integer', 'between:1900,2100'],
    ]);

    Book::create($validated);

    return redirect()
        ->route('books.index')
        ->with('success', 'Buku berhasil ditambahkan.');
```

resources/views/books/create.blade.php

```
@extends('layouts.app')

@section('content')
    <h1>Tambah Buku</h1>

    <form action="{{ route('books.store') }}" method="POST">
        @csrf

        <label for="title">Judul</label>
        <input
            id="title"
            type="text"
            name="title"
            value="{{ old('title') }}"
        >
        @error('title')
            <div class="error">{{ $message }}</div>
        @enderror

        <label for="author">Penulis</label>
        <input
            id="author"
            type="text"
            name="author"
            value="{{ old('author') }}"
        >
        @error('author')
            <div class="error">{{ $message }}</div>
        @enderror

        <label for="year">Tahun</label>
        <input
            id="year"
            type="number"
            name="year"
            value="{{ old('year') }}"
        >
        @error('year')
```

```

        <div class="error">{{ $message }}</div>
    @enderror

    <button type="submit">Simpan</button>
    <a href="{{ route('books.index') }}">Batal</a>
</form>
@endsection

```

Apa yang harus diamati?

Jika input tidak valid, Laravel mengembalikan pengguna ke form, membawa error validation, dan old input. Jika valid, data disimpan lalu browser diarahkan ke books.index dengan flash message.

Tahap 5 - Menguji Validation, Error, dan Old Input

Jangan hanya menguji jalur sukses. Praktikum dianggap benar jika jalur gagal juga menghasilkan perilaku yang tepat.

Kasus Uji	Input	Hasil yang Diharapkan
Valid	Judul, penulis, tahun 2024	Record baru tersimpan dan flash message tampil.
Judul kosong	title = kosong	Tidak tersimpan; error title tampil.
Penulis kosong	author = kosong	Tidak tersimpan; error author tampil.
Tahun teks	year = abc	Tidak tersimpan; error year tampil.
Tahun terlalu kecil	year = 1800	Tidak tersimpan karena outside range.
Old input	Salah satu field invalid	Field valid lain tetap berisi input sebelumnya.

Checkpoint 3

Masukkan data invalid minimal tiga kali. Pastikan tidak ada record invalid yang masuk ke database dan pengguna tidak perlu mengetik ulang field yang sudah benar.

Tahap 6 - Refactor Validation ke Form Request

Setelah alur create berjalan, pindahkan rules ke Form Request agar controller tetap ringkas dan aturan input terpusat.

Buat StoreBookRequest

```
php artisan make:request StoreBookRequest
```

app/Http/Requests/StoreBookRequest.php

```

<?php

namespace App\Http\Requests;

```

```

use Illuminate\Foundation\Http\FormRequest;

class StoreBookRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'author' => ['required', 'string', 'max:150'],
            'year' => ['required', 'integer', 'between:1900,2100'],
        ];
    }
}

```

BookController setelah refactor

```

use App\Http\Requests\StoreBookRequest;

public function store(StoreBookRequest $request)
{
    Book::create($request->validated());

    return redirect()
        ->route('books.index')
        ->with('success', 'Buku berhasil ditambahkan.');
```

Konsep

Form Request diproses sebelum method controller berjalan. `authorize()` menjawab “boleh melakukan request ini?”, sedangkan `rules()` menjawab “data apa yang dianggap valid?”.

Tahap 7 - UPDATE: Edit Form dan Update Data

Update juga terdiri dari dua request: `edit()` menampilkan data lama, sedangkan `update()` memvalidasi input baru dan mengubah record yang sama.

Buat UpdateBookRequest

```
php artisan make:request UpdateBookRequest
```

app/Http/Requests/UpdateBookRequest.php

```

<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateBookRequest extends FormRequest
{
    public function authorize(): bool
    {

```

```

        return true;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'author' => ['required', 'string', 'max:150'],
            'year' => ['required', 'integer', 'between:1900,2100'],
        ];
    }
}

```

Method edit() dan update()

```

use App\Http\Requests\UpdateBookRequest;

public function edit(Book $book)
{
    return view('books.edit', compact('book'));
}

public function update(UpdateBookRequest $request, Book $book)
{
    $book->update($request->validated());

    return redirect()
        ->route('books.index')
        ->with('success', 'Buku berhasil diperbarui.');
```

resources/views/books/edit.blade.php

```

@extends('layouts.app')

@section('content')
    <h1>Edit Buku</h1>

    <form action="{{ route('books.update', $book) }}" method="POST">
        @csrf
        @method('PUT')

        <label for="title">Judul</label>
        <input
            id="title"
            type="text"
            name="title"
            value="{{ old('title', $book->title) }}"
        >
        @error('title')
            <div class="error">{{ $message }}</div>
        @enderror

        <label for="author">Penulis</label>
        <input
            id="author"
            type="text"
            name="author"
            value="{{ old('author', $book->author) }}"
        >
        @error('author')
            <div class="error">{{ $message }}</div>
        @enderror

        <label for="year">Tahun</label>
        <input

```

```

        id="year"
        type="number"
        name="year"
        value="{{ old('year', $book->year) }}"
    >
    @error('year')
        <div class="error">{{ $message }}</div>
    @enderror

    <button type="submit">Simpan Perubahan</button>
    <a href="{{ route('books.index') }}">Batal</a>
</form>
@endsection

```

Method spoofing

HTML form hanya mengirim GET atau POST. `@method('PUT')` menambahkan hidden field `_method` sehingga Laravel Router memperlakukan request sebagai PUT.

Tahap 8 - DELETE: Menghapus Data secara Eksplisit

Operasi delete harus menggunakan HTTP DELETE, bukan GET link. Form tetap dikirim sebagai POST, kemudian `@method('DELETE')` memberi tahu router bahwa intent-nya adalah DELETE.

Method destroy()

```

public function destroy(Book $book)
{
    $book->delete();

    return redirect()
        ->route('books.index')
        ->with('success', 'Buku berhasil dihapus.');
```

resources/views/books/index.blade.php - versi akhir

```

@extends('layouts.app')

@section('content')
    <h1>Daftar Buku</h1>

    <p>
        <a href="{{ route('books.create') }}">Tambah Buku</a>
    </p>

    @if (session('success'))
        <p class="success">{{ session('success') }}</p>
    @endif

    <table>
        <thead>
            <tr>
                <th>Judul</th>
                <th>Penulis</th>
                <th>Tahun</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody>
            @forelse ($books as $book)
                <tr>

```

```

        <td>{{ $book->title }}</td>
        <td>{{ $book->author }}</td>
        <td>{{ $book->year }}</td>
        <td>
            <a href="{{ route('books.show', $book) }}">Detail</a>
            <a href="{{ route('books.edit', $book) }}">Edit</a>

            <form
                class="inline"
                action="{{ route('books.destroy', $book) }}"
                method="POST"
                onsubmit="return confirm('Hapus buku ini?')"
            >
                @csrf
                @method('DELETE')
                <button type="submit">Hapus</button>
            </form>
        </td>
    </tr>
    @empty
    <tr>
        <td colspan="4">Belum ada data buku.</td>
    </tr>
    @endforelse
</tbody>
</table>
@endsection

```

Checkpoint 4

Edit satu buku, pastikan record yang sama berubah. Setelah itu hapus satu buku dan pastikan record benar-benar hilang dari database serta flash message tampil.

Tahap 9 - BookController Versi Akhir

Setelah seluruh tahap selesai, BookController seharusnya ringkas karena validation sudah dipindahkan ke Form Request.

app/Http/Controllers/BookController.php - versi akhir

```

<?php

namespace App\Http\Controllers;

use App\Http\Requests\StoreBookRequest;
use App\Http\Requests\UpdateBookRequest;
use App\Models\Book;

class BookController extends Controller
{
    public function index()
    {
        $books = Book::latest()->get();

        return view('books.index', compact('books'));
    }

    public function create()
    {
        return view('books.create');
    }

    public function store(StoreBookRequest $request)
    {

```

```

        Book::create($request->validated());

        return redirect()
            ->route('books.index')
            ->with('success', 'Buku berhasil ditambahkan.');
```

```

    }

    public function show(Book $book)
    {
        return view('books.show', compact('book'));
    }

    public function edit(Book $book)
    {
        return view('books.edit', compact('book'));
    }

    public function update(UpdateBookRequest $request, Book $book)
    {
        $book->update($request->validated());

        return redirect()
            ->route('books.index')
            ->with('success', 'Buku berhasil diperbarui.');
```

```

    }

    public function destroy(Book $book)
    {
        $book->delete();

        return redirect()
            ->route('books.index')
            ->with('success', 'Buku berhasil dihapus.');
```

```

    }
}

```

Tiga Lapis Proteksi pada Form CRUD

Lapis	Pertanyaan	Implementasi
CSRF	Apakah request form berasal dari konteks sesi aplikasi?	@csrf
Validation	Apakah nilai input memenuhi aturan aplikasi?	\$request->validate() / Form Request
Mass assignment	Field mana yang boleh diisi secara massal?	\$fillable pada model

Ketiga lapis tersebut menyelesaikan masalah yang berbeda. Mengaktifkan salah satunya tidak menggantikan dua lapis lainnya.

Checklist Pengujian CRUD

- [] GET /books menampilkan semua buku.
- [] GET /books/create menampilkan form tambah.
- [] POST /books menyimpan input valid.
- [] Input invalid tidak masuk database.
- [] Error validation muncul di dekat field yang salah.
- [] old() mengembalikan input sebelumnya setelah validation gagal.

- [] GET /books/{book} menampilkan detail record yang benar.
- [] GET /books/{book}/edit menampilkan data lama.
- [] PUT /books/{book} memperbarui record yang sama.
- [] DELETE /books/{book} menghapus record.
- [] Flash message tampil setelah create, update, dan delete.
- [] route:list --path=books tidak menunjukkan route ganda yang tidak disengaja.

Troubleshooting Cepat

Gejala	Kemungkinan Penyebab	Yang Diperiksa
419 Page Expired	Token CSRF hilang/tidak valid	Pastikan @csrf ada pada form POST/PUT/DELETE.
405 Method Not Allowed	HTTP method tidak cocok	Periksa method form dan @method('PUT') / @method('DELETE').
MassAssignmentException / field tidak tersimpan	Field tidak diizinkan	Periksa \$fillable dan nama field.
404 pada books/{book}	Model binding tidak menemukan id	Pastikan record ada dan parameter route bernama {book}.
Error tidak tampil	Blade tidak membaca error bag	Periksa @error('field') dan name input.
Data lama hilang ketika validation gagal	Fallback old() tidak tepat	Gunakan old('field') atau old('field', \$book->field).

Challenge Akhir - Tambahkan ISBN

Kerjakan tanpa mengubah struktur CRUD yang sudah berjalan. Challenge dianggap selesai jika semua kriteria berikut terpenuhi.

1. Buat migration baru untuk menambahkan kolom isbn pada tabel books.
2. Kolom isbn harus diwajibkan dan unik.
3. Tambahkan field isbn pada form create dan edit.
4. Tambahkan isbn ke model agar dapat diisi melalui create()/update().
5. Tambahkan validation ISBN pada StoreBookRequest.
6. Pada UpdateBookRequest, ISBN milik record yang sedang diedit tidak boleh dianggap duplikat.
7. Tampilkan ISBN pada halaman daftar atau detail buku.
8. Pastikan flash message tetap tampil setelah create, update, dan delete.

Perintah awal challenge

```
php artisan make:migration add_isbn_to_books_table --table=books
```

Petunjuk jika buntu

Untuk aturan unique pada update, gunakan Rule::unique(...) dan abaikan model Book yang sedang diproses. Periksa kembali nama parameter resource route: book.

Kriteria Selesai Challenge

- ☐ Migration up() dan down() bekerja tanpa error.
- ☐ Dua buku tidak dapat memiliki ISBN yang sama.
- ☐ Saat mengedit buku tanpa mengubah ISBN, validation tetap lolos.
- ☐ Field ISBN mempertahankan old input saat validation gagal.
- ☐ Tidak ada perubahan route CRUD yang tidak diperlukan.

Refleksi Praktikum

Jawab singkat setelah seluruh checkpoint selesai.

Mengapa create() dan store() dipisahkan menjadi dua action?

.....

Mengapa @csrf tidak dapat menggantikan validation?

.....

Mengapa validation client-side tidak cukup untuk melindungi data?

.....

Apa fungsi @method('PUT') dan @method('DELETE')?

.....

Mengapa Book::create(\$request->all()) sebaiknya dihindari pada input pengguna?

.....

Apa manfaat memindahkan validation dari controller ke Form Request?

.....

Checklist Penyelesaian Praktikum

- ☐ Semua checkpoint 1-4 berhasil.
- ☐ CRUD lengkap dapat didemonstrasikan dari browser.
- ☐ Validation gagal tidak menghasilkan record baru/perubahan data.
- ☐ CSRF dan method spoofing digunakan pada form yang sesuai.
- ☐ Model memiliki mass-assignment whitelist.
- ☐ StoreBookRequest dan UpdateBookRequest digunakan.
- ☐ Challenge ISBN selesai.
- ☐ Project dapat dijalankan ulang tanpa error setelah browser dan server direstart.

Referensi

- **Laravel 13 - Controllers dan Resource Controllers:** <https://laravel.com/docs/13.x/controllers>
- **Laravel 13 - Routing:** <https://laravel.com/docs/13.x/routing>
- **Laravel 13 - Validation dan Form Request:** <https://laravel.com/docs/13.x/validation>
- **Laravel 13 - CSRF Protection:** <https://laravel.com/docs/13.x/csrf>
- **Laravel 13 - Eloquent ORM dan Mass Assignment:** <https://laravel.com/docs/13.x/eloquent>

Ringkasan akhir

CRUD Laravel yang baik bukan hanya “empat operasi yang jalan”. Setiap operasi harus memiliki HTTP method dan route yang tepat, validation sebelum data dipercaya, pembatasan mass assignment, serta feedback yang jelas kepada pengguna.